Post-contenido — Operaciones con cadenas y aritmética

Arquitectura de Computadores · Unidad 8

Estudiante: González López

Repositorio: TrabajosArqui — post8-u8

PDF: Gonzalez Lopez_Post8_U8.pdf

Contenido y entorno

Esta actividad integra cuatro programas de cadenas y cuatro de aritmética, fuentes NASM de 16 bits, ocho binarios `.COM` y este documento. Los fuentes se ensamblan con NASM desde Windows usando `build.bat`; los `.COM` se ejecutan en DOSBox 0.74-3. En los programas de cadena se inicializa `ES=DS` y se ejecuta `CLD` explícitamente antes de las instrucciones que usan SI/DI. El contador de una operación MOVS cuenta elementos, no bytes: MOVSW copia dos bytes por iteración. El programa de búsqueda calcula el índice base cero y el de comparación usa longitudes conocidas.

Desde PowerShell, en esta carpeta:

```
.\build.bat
```

En DOSBox, montar la carpeta del proyecto (ajustar la ruta a la ubicación local):

```
mount c "C:\ruta\al\repositorio\unidad-8\post8-u8"
c:
bin\post1.com
bin\post1c.com
bin\post1d.com
bin\post1e.com
bin\post2.com
bin\post2b.com
bin\post2c.com
bin\post2d.com
```

Los programas `post1.asm` y `post2b.asm` demuestran además un manejo relevante del estado: MOVSB se verifica con un buffer de longitud impar, y el acarreo de DAA se captura antes de llamar a DOS, porque las llamadas posteriores pueden modificar FLAGS.

Programas y resultados esperados

Fuente | Operación | Resultado esperado

`src/post1.asm` | REP MOVSB y REP MOVSW con byte residual | `HOLA, MUNDO!`

`src/post1c.asm` | REPNE SCASB, búsquedas de d y z | Índice 13 y ausencia de z

`src/post1d.asm` | REPE CMPSB con longitud fija | `Iguales.` y `Diferentes.`

`src/post1e.asm` | LODSB/STOSB y conversión ASCII a mayúsculas | `HOLA, MUNDO!`

`src/post2.asm` | ADD/ADC y SUB/SBB, dos palabras | `0003:0000` y `0001:FFFF`

`src/post2b.asm` | DAA y DAS con BCD empaquetado | 85, 45 y 00 con CF=1 en 99+01

`src/post2c.asm` | MUL/AAM y DIV, operandos de un dígito | 7×8=56; 9/3=3 resto 0

`src/post2d.asm` | AAA y AAS con BCD desempaquetado | 7+8=15; 13−8=5

Parte 1 — Cadena

`post1.asm` copia 13 bytes (incluido el terminador) primero con `REP MOVSB` y después con seis `MOVSW` más el byte restante. `post1c.asm` limita `REPNE SCASB` al tamaño de la cadena y usa ZF para distinguir éxito de agotamiento de CX. `post1d.asm` aplica `REPE CMPSB` a cadenas literales de ocho bytes. `post1e.asm` hace el recorrido con `LODSB`, transforma únicamente a–z y almacena cada resultado con `STOSB`. DF controla el sentido de incremento de SI/DI; por eso los ejemplos establecen DF=0 mediante CLD.

Parte 2 — Aritmética

`post2.asm` verifica ADD/ADC para una suma de 32 bits y SUB/SBB para una resta, usando las palabras bajas y altas. `post2b.asm` corrige suma y resta BCD empaquetadas con DAA/DAS y conserva CF inmediatamente después de DAA. `post2c.asm` convierte dígitos ASCII, multiplica con MUL y usa AAM para obtener decenas y unidades; DIV produce cociente y residuo y se comprueba el divisor cero. `post2d.asm` ajusta dígitos desempaquetados con AAA/AAS. Estos opcodes BCD se ejecutan en modo real/compatibilidad, no en modo largo de 64 bits.

Decisiones técnicas

1. ¿REP MOVSB o un bucle manual con LODSB/STOSB?

Para una copia sin transformación, `REP MOVSB` expresa el recorrido completo en una instrucción: CX controla las repeticiones y los punteros se actualizan automáticamente según DF. La CPU puede implementar esa operación repetida con mecanismos optimizados; el bucle manual, en cambio, debe cargar, almacenar, decrementar o probar el contador y volver al inicio en cada iteración. La diferencia real depende de la microarquitectura, tamaño y alineación, por lo que no se afirma una mejora universal de rendimiento, pero el código repetido evita el costo explícito de un salto de software por byte. `post1e.asm` abandona REP MOVSB porque debe inspeccionar AL y convertir cada letra minúscula antes de almacenarla: MOVSB no ofrece un punto de intervención por byte. Para copiar un bloque largo sin modificarlo se preferiría inicialmente una operación REP MOVS y se mediría en el procesador objetivo; se reservaría LODSB/STOSB para casos que requieren una decisión o transformación por elemento.

2. ¿CX fijo o comprobación de terminador?

Un contador fijo es seguro cuando la longitud del objeto está conocida y validada; ese es el caso de `post1d.asm`, donde ambas cadenas tienen exactamente ocho bytes. Si se comparara con CX=8 una cadena terminada tras cinco caracteres con otra de ocho, CMPS podría leer los tres bytes que siguen al cero de la corta. Esos bytes pueden pertenecer a otra variable o a memoria no destinada a la cadena, y podrían producir una comparación falsa o una lectura inválida. Para longitudes desconocidas se puede recorrer con un bucle manual que lea un byte de cada origen, compare los dos, finalice si son distintos o si alguno es cero, y actualice ambos índices; no se debe confiar en que CMPS reconozca automáticamente el terminador. Para SCASB, la alternativa es calcular o limitar explícitamente el máximo accesible y buscar el cero dentro de ese límite. CX fijo es eficiente y correcto con longitudes conocidas; para cadenas arbitrarias se requiere terminador o longitud explícita para evitar salir del objeto.

3. ¿DAA/DAS o AAA/AAS según la representación?

La selección depende del formato de entrada y no solamente de que el dato represente un número decimal. DAA y DAS operan sobre un byte empaquetado con dos dígitos, uno en cada nibble, y corrigen el resultado de ADD o SUB usando AF/CF y las condiciones del nibble. AAA y AAS se aplican después de sumar o restar dígitos desempaquetados y dejan el dígito de unidades en el nibble bajo de AL, con el acarreo o préstamo decimal en AH. Si 47h representa los dos dígitos empaquetados “47”, AAA no es apropiada: interpretaría el nibble bajo como un dígito y puede limpiar el nibble alto, destruyendo el 4 de las decenas. En cambio, restar 30h de un carácter ASCII genera un dígito de un byte (por ejemplo 07h); para ajustar 07h+08h=0Fh corresponde AAA, que produce AH:AL=01:05. DAA trataría el nibble alto cero como parte del número empaquetado y no codificaría correctamente las dos posiciones del resultado. Por ello, las rutinas deben documentar su formato y emparejarlo con su

instrucción de ajuste.

4. ¿ADC fijo o una cadena ADC en bucle?

`post2.asm` usa ADD para la palabra menos significativa y ADC para la palabra alta: para una cifra de dos palabras esta secuencia fija es compacta y evita el control adicional de un bucle. Si el tamaño fuera arbitrario, se recorrerían arreglos de palabras desde el índice cero, que representa el menor peso; primero se suman los operandos con ADD, y en cada posición posterior se emplea ADC para incorporar CF que dejó la suma anterior. El resultado se almacena en la misma posición y puede requerirse una palabra adicional para el acarreo final. Un bucle genérico hace posible reutilizar la rutina para 48, 64 o más bits sin duplicar instrucciones por tamaño y reduce errores al ampliar precisión, a costa de preparar índices y contador y ejecutar una rama de control. Es una compensación semejante a la de la Parte 1: la forma fija puede ser más rápida y simple para un tamaño específico, mientras el bucle aporta mantenibilidad y flexibilidad. En código crítico conviene medir y elegir con base en el tamaño real.

Validación y capturas

Los ocho fuentes se ensamblaron con NASM 2.16.03. La sesión de DOSBox 0.74-3 aceptó la secuencia de ejecución de los programas no interactivos y cerró sin error; todavía falta revisar visualmente sus salidas y probar la entrada interactiva de `post2c.com`. Las capturas auténticas y el estado de cada uno de los trece checkpoints se registran en `capturas/README.md`; no se incluyen imágenes sintéticas. Para completar la calculadora, probar `7`, `8`, `*`; `9`, `3`, `/`; y cualquier dígito, `0`, `/` para cubrir la división por cero.